NATIONAL ECONOMICS UNIVERSITY

Mathematics Faculty of Economics

FINAL PROJECT

DATABASE MANAGEMENT SYSTEMS

PROJECT 13

PERSONAL FINANCE MANAGEMENT SYSTEM

Student : **Nguyen Phuong Ngan**

Class : **DSEB 66B**

Student ID : **11245914**

Instructor : **PhD. Tran Hung**

Hanoi, 5/2026

PROJECT 13: PERSONAL FINANCE MANAGEMENT SYSTEM

DATCOM Lab
NEU-College of Technology
National Economics University
Email: hung.tran@neu.edu.vn

Project Objective

The objective of this project is to develop a system that enables users to effectively manage personal income and expenses, track financial activities, analyze spending habits, and optimize savings through data-driven reports and alerts.

Database and Programming Languages

- **Database Management System:** MySQL
- **Programming Language:** Python

Detailed Project Requirements

System Analysis and Requirements

- Manage personal financial data including income, expenses, bank accounts, and spending categories.
- Allow users to monitor their financial health through detailed summaries and visual reports.

Main Functionalities

- User profile management.
- Income and expense entry, update, and categorization.
- Bank account tracking and balance history.
- Daily, monthly, and yearly financial summaries.
- Budget planning and spending limit alerts.
- Reporting on trends and category-wise expenditures.

Database Design and Implementation

1. **Data Model Design**
    - Create an ER diagram modeling users, income, expenses, and accounts.
    - Convert to relational schema with PKs, FKs, and necessary constraints.
2. **Table Structures**
    - Users (UserID, UserName, Email, PhoneNumber)
    - Income (IncomeID, UserID, Amount, IncomeDate, Description)
    - ExpenseCategories (CategoryID, CategoryName)
    - Expenses (ExpenseID, UserID, CategoryID, Amount, ExpenseDate, Description)
    - BankAccounts (AccountID, UserID, BankName, Balance)
3. **Sample Data**
    - Populate each table with 5–10 representative records.
    - Create a database schema diagram using MySQL Workbench.

Advanced Database Objects

- **Indexes:** Enhance performance for user queries and expense lookups.
- **Views:** Create monthly income/expense summaries, and category-wise spending views.
- **Stored Procedures:** Automate balance updates and monthly closure operations.
- **User Defined Functions:** Compute total income, total expenses, or budget status.
- **Triggers:** Automatically update bank account balance upon income or expense entry.

Database Security and Administration

- Secure user information with authentication roles.
- Implement access controls to protect personal financial data.
- Ensure backup and recovery procedures are in place.
- Optimize query performance for report generation.

Python Application Development

- **Database Connection:** Use `mysql-connector-python` or `SQLAlchemy`.
- **Finance Tracker:** Build scripts for users to input transactions, track expenses, and view summaries.
- **Reporting:** Generate graphical and tabular financial reports.
- **User Interface:** Optional CLI or GUI to enhance user experience.

Deliverables

- A printed comprehensive report including database design, implementation details, and screenshots. Submit your report into LMS. First pages of your report must attach this PDF file.
- Link to publish Github source code and video presenting on Youtube
- SQL schema, sample data scripts, ER diagrams, and Python code.
- Demonstration of functionalities such as adding income, managing expenses, and generating reports.

Conclusion and Recommendations

- Summarize key achievements of the system.
- Recommend features such as saving goals, predictive analytics, or integration with mobile banking.

References

- Include all resources, documentation, and libraries used during the project.

Contents

1 Problem Understanding and Requirement Analysis

1.1 Problem Statement

Managing personal finances is a persistent challenge for individuals, particularly young professionals and students in Vietnam, where cash-based transactions remain common and awareness of structured budgeting is limited. Without a centralised system, people frequently lose track of their spending patterns, miss budget thresholds, and struggle to identify which expense categories are draining their income.

This project addresses that gap by building a **Personal Finance Management System (PFMS)** – a desktop application backed by a relational database – that enables users to record income and expenses, monitor bank account balances in real time, set spending budgets per category, and analyse their financial behaviour through visual reports.

1.2 Objectives

The system is designed to achieve the following objectives:

1. Provide a persistent, structured data store for all financial transactions using MySQL, exploiting advanced database features (triggers, stored procedures, views, UDFs) to enforce business rules at the database layer.
2. Deliver an intuitive desktop graphical interface (CustomTkinter) that allows non-technical users to record transactions, manage budgets, and view analytics without writing any SQL.
3. Demonstrate the practical value of database-level automation: bank account balances must always reflect the sum of all income minus all expenses, enforced by a full lifecycle set of six triggers rather than application code, so data integrity is guaranteed regardless of how the database is accessed.
4. Produce actionable financial insights through two reporting modes: a **Performance Snapshots** tab presenting daily, monthly, and yearly numerical summaries side-by-side; and a **Visual Analytics** tab containing three interactive chart types – income-versus-expenses bar chart, category-spending pie chart, and cumulative balance trend line chart – with hover tooltips displaying exact VND values.

1.3 Scope

In scope: User profile management (create and edit); bank account tracking; income and expense entry, editing, and deletion with full trigger-driven balance adjustment; monthly budget setting, updating, and deletion; three graphical reports with interactive tooltips; daily, monthly, and yearly performance snapshots; database security via role-restricted MySQL user; automated balance updates via six triggers.

Out of scope: Mobile application; integration with real banking APIs; multi-currency support; cloud synchronisation; collaborative (multi-device) access.

1.4 Inputs and Outputs

Table 1: System Inputs and Outputs

Direction	Description
Inputs	User profile data (name, email, phone); bank account details (bank name, initial balance); income records (amount, date, account, description); expense records (amount, category, account, date, description); budget limits (category, month, maximum amount); edit and delete requests for existing income, expense, and budget records.
Outputs	Real-time account balance (updated automatically by trigger); daily income/expense totals via `CURDATE()` query; monthly income/expense summary; yearly net savings aggregate; budget utilisation status with WARNING/EXCEEDED alerts; three interactive matplotlib charts with hover tooltips embedded in the UI; monthly closure snapshot via stored procedure.

1.5 Functional Requirements

Table 2: Functional Requirements

ID	Priority	Requirement
FR-01	Must	The system shall maintain a Users table with unique email addresses.
FR-02	Must	Each user may have multiple bank accounts; balance must equal the running total of income minus expenses for that account.
FR-03	Must	Every income insertion shall automatically increment the linked bank account balance (via trigger).
FR-04	Must	Every expense insertion shall automatically decrement the linked bank account balance (via trigger); balance must not fall below zero.
FR-05	Must	Users shall be able to set a monthly spending budget per expense category; budgets may also be deleted when no longer needed.
FR-06	Must	The system shall calculate budget utilisation percentage and flag status as OK / WARNING ($\geq$80%) / EXCEEDED ($\geq$100%).
FR-07	Must	Monthly income-vs-expense reports shall be generated using a database view.
FR-08	Must	A stored procedure shall return a full monthly financial report for a user.
FR-09	Should	UDFs shall encapsulate total-expense and budget-usage calculations for reuse in SQL queries.
FR-10	Should	The Python GUI shall provide four screens: Dashboard, Transactions, Reports, Budgets.
FR-11	Must	The system shall provide daily financial summaries (today's income and expenses) and yearly aggregates for the selected year, displayed alongside monthly figures in the Reports screen.
FR-12	Should	Users shall be able to perform full CRUD operations on income and expense records; edits and deletions shall trigger automatic balance adjustments via database triggers.
FR-13	Should	Users shall be able to create new expense categories inline from the Add Expense form, and update their own profile information from the sidebar.

1.6 Non-Functional Requirements

- **Data integrity:** All foreign key relationships enforce referential integrity with `ON DELETE CASCADE` where appropriate. Check constraints prevent negative amounts and malformed email addresses.
- **Security:** The Python application connects to MySQL using a restricted user (`finance_app`) that holds only SELECT, INSERT, UPDATE, DELETE (on transaction tables), and EXECUTE privileges – no DROP or structural DDL.
- **Performance:** Composite indexes on `(UserID, ExpenseDate)` and `(UserID, IncomeDate)` ensure sub-millisecond filtering on typical dataset sizes.
- **Maintainability:** Code is split into `db_connection.py`, `models.py`, `reports.py`, and `main.py`, following a separation-of-concerns architecture. The UI layer contains zero SQL strings.

2 Design and Technical Soundness

2.1 System Architecture

The Personal Finance Management System follows a structured three-layer architecture to ensure modularity and security. This design separates the user interface from the data management logic, as illustrated in Figure 1.

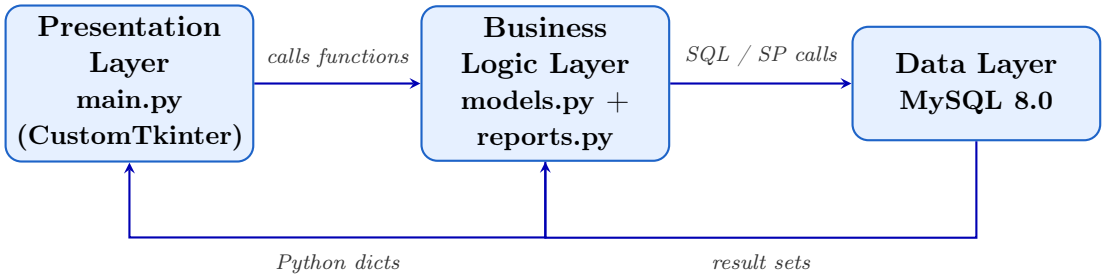


Figure 1: System architecture showing the interaction between UI, logic, and database layers.

Design rationale: The UI layer is strictly decoupled from the database, meaning it never contains raw SQL strings. All data interactions are routed through `models.py` using parameterised queries or stored procedures. Chart generation is handled exclusively by `reports.py`, which calls `models.py` functions to retrieve data and returns `matplotlib Figure` objects that `main.py` embeds via `FigureCanvasTkAgg`. Interactive tooltips are implemented by registering `motion_notify_event` callbacks in `main.py` against hidden annotation objects pre-built inside each chart. This architecture makes SQL injection structurally impossible from the presentation layer and ensures that schema changes only necessitate updates in a single localised file.

2.2 Entity-Relationship Design

The data model comprises six entities. The cardinalities and relationships are summarised below:

Table 3: Entity Relationships

Entity A	Entity B	Relationship
Users	BankAccounts	One-to-Many (a user owns multiple accounts)
Users	Income	One-to-Many (a user records multiple income entries)
Users	Expenses	One-to-Many (a user records multiple expenses)
Users	Budgets	One-to-Many (a user sets budgets per month/category)
BankAccounts	Income	One-to-Many (income is deposited into one account)
BankAccounts	Expenses	One-to-Many (expenses are deducted from one account)
ExpenseCategories	Expenses	One-to-Many (each expense belongs to one category)
ExpenseCategories	Budgets	One-to-Many (budgets are set per category)

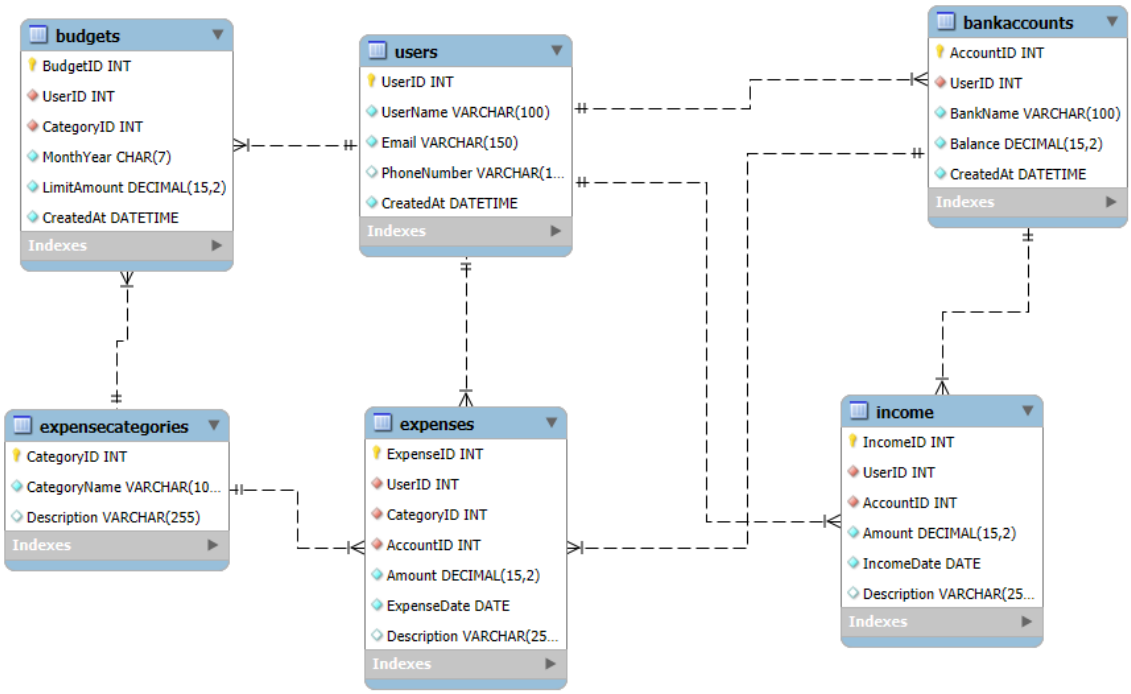


Figure 2: ER Diagram

2.3 Relational Schema

2.3.1 Table: Users

Table 4: Users Table Structure

Column	Data Type	Constraints	Description
UserID	INT	PK, AUTO_INCREMENT	Surrogate primary key
UserName	VARCHAR(100)	NOT NULL	Full display name
Email	VARCHAR(150)	NOT NULL, UNIQUE	Login identifier; CHECK format
PhoneNumber	VARCHAR(15)	–	Optional contact number
CreatedAt	DATETIME	DEFAULT NOW()	Account creation timestamp

2.3.2 Table: BankAccounts

Table 5: BankAccounts Table Structure

Column	Data Type	Constraints	Description
AccountID	INT	PK, AUTO_INCREMENT	Surrogate primary key
UserID	INT	FK → Users, NOT NULL	Account owner
BankName	VARCHAR(100)	NOT NULL	Name of the bank
Balance	DECIMAL(15,2)	DEFAULT 0, CHECK ≥0	Current balance; auto-updated by triggers
CreatedAt	DATETIME	DEFAULT NOW()	Account opening timestamp

2.3.3 Table: Income

Table 6: Income Table Structure

Column	Data Type	Constraints	Description
IncomeID	INT	PK, AUTO_INCREMENT	Surrogate primary key
UserID	INT	FK → Users	Income recipient
AccountID	INT	FK → BankAccounts	Target account
Amount	DECIMAL(15,2)	NOT NULL, CHECK > 0	Income amount in VND
IncomeDate	DATE	NOT NULL	Date received
Description	VARCHAR(255)	–	Optional note

2.3.4 Table: ExpenseCategories

Table 7: ExpenseCategories Table Structure

Column	Data Type	Constraints	Description
CategoryID	INT	PK, AUTO_INCREMENT	Surrogate primary key
CategoryName	VARCHAR(100)	NOT NULL, UNIQUE	e.g. Food & Dining, Transport
Description	VARCHAR(255)	–	Optional category description

2.3.5 Table: Expenses

Table 8: Expenses Table Structure

Column	Data Type	Constraints	Description
ExpenseID	INT	PK, AUTO_INCREMENT	Surrogate primary key
UserID	INT	FK → Users	Spender
CategoryID	INT	FK → ExpenseCategories	Spending classification
AccountID	INT	FK → BankAccounts	Source account
Amount	DECIMAL(15,2)	NOT NULL, CHECK > 0	Expense amount in VND
ExpenseDate	DATE	NOT NULL	Date of expense
Description	VARCHAR(255)	–	Optional note

2.3.6 Table: Budgets

Table 9: Budgets Table Structure

Column	Data Type	Constraints	Description
BudgetID	INT	PK, AUTO_INCREMENT	Surrogate primary key
UserID	INT	FK → Users	Budget owner
CategoryID	INT	FK → ExpenseCategories	Budget category
MonthYear	CHAR(7)	NOT NULL (format YYYY-MM)	Budget month
LimitAmount	DECIMAL(15,2)	NOT NULL, CHECK > 0	Monthly spending ceiling
CreatedAt	DATETIME	DEFAULT NOW()	Creation timestamp

2.4 Advanced Database Objects

2.4.1 Indexes

Three composite indexes were created to optimise the most frequent query patterns:

```sql
-- Speeds up: "Show expenses for user X in month Y"
CREATE INDEX idx_expenses_user_date
    ON Expenses(UserID, ExpenseDate);

-- Speeds up: "Show income for user X in month Y"
CREATE INDEX idx_income_user_date
    ON Income(UserID, IncomeDate);

-- Speeds up: budget utilisation checks per user/category/month
CREATE INDEX idx_budgets_user_cat_month
    ON Budgets(UserID, CategoryID, MonthYear);
```

Listing 1: Index Definitions

Justification: Without these indexes, every monthly filter would require a full table scan. With a B-tree index on `(UserID, ExpenseDate)`, MySQL can seek directly to the relevant rows in $O(\log n)$ time.

2.4.2 Views

The system utilizes database views to abstract complex `JOIN` logic and multi-level aggregations into reusable virtual tables. This ensures that the Python application remains database-agnostic for its reporting needs, fetching pre-calculated results via simple `SELECT` statements.

1. Monthly Financial Overview (`vw_monthly_summary`) This view provides the primary data for the monthly bar charts. Because MySQL 8.0 does not natively support `FULL OUTER JOIN`, a `UNION` of a `LEFT JOIN` and a `RIGHT JOIN` is implemented to ensure months with only income or only expenses are never omitted.

```sql
CREATE OR REPLACE VIEW vw_monthly_summary AS
SELECT
    u.UserID, u.UserName, summary.MonthYear,
    summary.TotalIncome, summary.TotalExpenses,
    (summary.TotalIncome - summary.TotalExpenses) AS NetSavings
FROM Users u
JOIN (
    SELECT
        COALESCE(inc.UserID, exp.UserID) AS UserID,
        COALESCE(inc.MonthYear, exp.MonthYear) AS MonthYear,
        COALESCE(inc.TotalIncome, 0) AS TotalIncome,
        COALESCE(exp.TotalExpenses, 0) AS TotalExpenses
    FROM (
        -- Aggregate Income by User and Month
        SELECT UserID, DATE_FORMAT(IncomeDate, '%Y-%m')
            AS MonthYear, SUM(Amount) AS TotalIncome
        FROM Income GROUP BY UserID, MonthYear
    ) inc
    LEFT JOIN (
        -- Aggregate Expenses by User and Month
        SELECT UserID, DATE_FORMAT(ExpenseDate, '%Y-%m')
            AS MonthYear, SUM(Amount) AS TotalExpenses
        FROM Expenses GROUP BY UserID, MonthYear
    ) exp ON inc.UserID = exp.UserID
            AND inc.MonthYear = exp.MonthYear

    UNION -- Ensure months with only expenses are included

    SELECT
        COALESCE(inc.UserID, exp.UserID) AS UserID,
        COALESCE(inc.MonthYear, exp.MonthYear) AS MonthYear,
        COALESCE(inc.TotalIncome, 0) AS TotalIncome,
        COALESCE(exp.TotalExpenses, 0) AS TotalExpenses
    FROM (
        SELECT UserID, DATE_FORMAT(IncomeDate, '%Y-%m')
            AS MonthYear, SUM(Amount) AS TotalIncome
        FROM Income GROUP BY UserID, MonthYear
    ) inc
    RIGHT JOIN (
        SELECT UserID, DATE_FORMAT(ExpenseDate, '%Y-%m')
            AS MonthYear, SUM(Amount) AS TotalExpenses
        FROM Expenses GROUP BY UserID, MonthYear
```

```
    ) exp ON inc.UserID = exp.UserID
            AND inc.MonthYear = exp.MonthYear
) summary ON u.UserID = summary.UserID;
```

Listing 2: vw_monthly_summary View (excerpt)

2. Category-wise Spending (`vw_category_spending`) This view breaks down expenditures by category for each user per month. It simplifies data retrieval for `reports.py` to generate pie charts without re-calculating sums in Python.

```
CREATE OR REPLACE VIEW vw_category_spending AS
SELECT
    e.UserID, u.UserName, ec.CategoryID, ec.CategoryName,
    DATE_FORMAT(e.ExpenseDate, '%Y-%m')      AS MonthYear,
    SUM(e.Amount)                            AS TotalSpent,
    COUNT(e.ExpenseID)                       AS TransactionCount
FROM Expenses e
JOIN Users u              ON e.UserID     = u.UserID
JOIN ExpenseCategories ec ON e.CategoryID = ec.CategoryID
GROUP BY e.UserID, u.UserName, ec.CategoryID,
         ec.CategoryName, MonthYear;
```

Listing 3: vw_category_spending View

3. Budget Performance Tracking (`vw_budget_status`) This view compares budget limits to actual expenditures and encapsulates the system's core business logic – categorising financial health into three tiers: `OK`, `WARNING`, and `EXCEEDED`.

```
CREATE OR REPLACE VIEW vw_budget_status AS
SELECT
    b.BudgetID, b.UserID, b.CategoryID, u.UserName,
    ec.CategoryName, b.MonthYear, b.LimitAmount,
    COALESCE(spent.TotalSpent, 0) AS SpentAmount,
    b.LimitAmount - COALESCE(spent.TotalSpent, 0)
        AS RemainingAmount,
    ROUND(
        COALESCE(spent.TotalSpent, 0) / b.LimitAmount * 100, 2)
            AS UsagePercent,
    CASE
        WHEN COALESCE(spent.TotalSpent, 0) >= b.LimitAmount
            THEN 'EXCEEDED'
        WHEN COALESCE(spent.TotalSpent, 0) >= b.LimitAmount*0.8
            THEN 'WARNING'
        ELSE 'OK'
    END AS BudgetStatus
FROM Budgets b
JOIN Users u              ON b.UserID     = u.UserID
JOIN ExpenseCategories ec ON b.CategoryID = ec.CategoryID
LEFT JOIN (
    -- Subquery to get total spent per user/category/month
    SELECT UserID, CategoryID, DATE_FORMAT(ExpenseDate, '%Y-%m')
        AS MonthYear, SUM(Amount) AS TotalSpent
    FROM Expenses GROUP BY UserID, CategoryID, MonthYear
```

```
) spent ON b.UserID = spent.UserID
        AND b.CategoryID = spent.CategoryID
        AND b.MonthYear = spent.MonthYear;
```

Listing 4: vw_budget_status View (excerpt)

Table 10: Database Views Technical Analysis

View Name	Technical Purpose and Application Link
`vw_monthly_summary`	Aggregates net savings by simulating a `FULL JOIN`. Used by `reports.py` for the bar chart and by `get_yearly_summary()` for yearly aggregation.
`vw_category_spending`	Groups transactions by category per month. Provides the data source for the category pie chart.
`vw_budget_status`	Computes real-time budget utilisation percentages and status labels. Powers both the Budgets screen and the Dashboard alerts panel.

Summary of Database Views Technical Analysis: By leveraging subqueries within these views, the system avoids Cartesian Product errors where joining multiple transaction rows might lead to inflated sums. The use of `COALESCE` ensures data robustness, allowing the system to handle months or categories with zero activity without returning `NULL` values to the Python layer.

2.4.3 User-Defined Functions

Three UDFs encapsulate repetitive calculations that can be integrated into `SELECT` statements and other database objects.

```
CREATE FUNCTION fn_get_budget_usage_percent(
    p_UserID        INT,
    p_CategoryID    INT,
    p_MonthYear     CHAR(7)
)
RETURNS DECIMAL(5, 2)
READS SQL DATA
DETERMINISTIC
BEGIN
    DECLARE v_spent     DECIMAL(15, 2) DEFAULT 0.00;
    DECLARE v_limit     DECIMAL(15, 2) DEFAULT 0.00;
    DECLARE v_start_date DATE;
    SET v_start_date = STR_TO_DATE(CONCAT(p_MonthYear, '-01'),
                                   '%Y-%m-%d');

    -- Get actual spending this month in this category
    SELECT COALESCE(SUM(Amount), 0)
    INTO   v_spent
    FROM   Expenses
    WHERE  UserID     = p_UserID
        AND  CategoryID = p_CategoryID
        AND  ExpenseDate >= v_start_date
```

```
        AND  ExpenseDate <  DATE_ADD(v_start_date,
                                     INTERVAL 1 MONTH);

    -- Get the budget limit
    SELECT COALESCE(LimitAmount, 0)
    INTO   v_limit
    FROM   Budgets
    WHERE  UserID     = p_UserID
        AND  CategoryID = p_CategoryID
        AND  MonthYear  = p_MonthYear
    LIMIT 1;

    -- Avoid division by zero
    IF v_limit = 0 THEN
        RETURN 0.00;
    END IF;

    RETURN ROUND((v_spent / v_limit) * 100, 2);
END;
```

Listing 5: UDF: fn_get_budget_usage_percent

Table 11: User-Defined Functions Analysis

Function	Returns	Description
`fn_get_total_income` (UserID, MonthYear)	DECIMAL	Calculates total income for a user in a specific month; used in stored procedures for net savings calculations.
`fn_get_total_expenses` (UserID, MonthYear)	DECIMAL	Sum of all expenses for a user in a given month; called inside `sp_monthly_report` for automated balance validation.
`fn_get_budget_usage_percent` (UserID, CategoryID, MonthYear)	DECIMAL	Calculates the ratio of actual spending to budget limit (0–100+); used to trigger UI alerts in the Dashboard.

2.4.4 Stored Procedures

```
CREATE PROCEDURE sp_monthly_report(
    IN p_UserID INT, IN p_MonthYear CHAR(7))
BEGIN
    -- Result set 1: Summary
    SELECT
        p_MonthYear AS ReportMonth,
        fn_get_total_income(p_UserID, p_MonthYear)
            AS TotalIncome,
        fn_get_total_expenses(p_UserID, p_MonthYear)
            AS TotalExpenses,
        (fn_get_total_income(p_UserID, p_MonthYear) -
            fn_get_total_expenses(p_UserID, p_MonthYear))
                AS NetSavings;

```

```
    -- Result set 2: Category breakdown
    SELECT CategoryName, SpentAmount, UsagePercent, BudgetStatus
    FROM vw_budget_status
    WHERE UserID = p_UserID AND MonthYear = p_MonthYear
    ORDER BY SpentAmount DESC;
END;
```

Listing 6: Stored Procedure: sp_monthly_report

Table 12: Stored Procedures Technical Analysis

Procedure	Technical Description
`sp_monthly_report` (UserID, MonthYear)	Returns two result sets: (1) overall income vs expense summary using UDFs; (2) category-level spending with budget status from `vw_budget_status`.
`sp_close_month` (UserID, MonthYear)	Computes a month-end financial snapshot for audit or historical records, calling both income and expense UDFs.

2.4.5 Triggers

The system employs six triggers to automate `BankAccounts.Balance` updates whenever an income or expense record is created, modified, or deleted.

```
CREATE TRIGGER trg_income_after_insert
AFTER INSERT ON Income
FOR EACH ROW
BEGIN
    UPDATE BankAccounts
    SET    Balance = Balance + NEW.Amount
    WHERE  AccountID = NEW.AccountID;
END;
```

Listing 7: Trigger: trg_income_after_insert

The UPDATE triggers handle two distinct cases to support account transfers:

```
CREATE TRIGGER trg_expense_after_update
AFTER UPDATE ON Expenses
FOR EACH ROW
BEGIN
    -- Case 1: Account changed - reverse old, apply to new
    IF OLD.AccountID <> NEW.AccountID THEN
        UPDATE BankAccounts
        SET Balance = Balance + OLD.Amount
        WHERE AccountID = OLD.AccountID;

        UPDATE BankAccounts
        SET Balance = Balance - NEW.Amount
        WHERE AccountID = NEW.AccountID;

    -- Case 2: Same account, amount changed
    ELSEIF OLD.Amount <> NEW.Amount THEN
```

```
        UPDATE BankAccounts
        SET Balance = Balance + (OLD.Amount - NEW.Amount)
        WHERE AccountID = NEW.AccountID;
    END IF;
END;
```

Listing 8: Trigger: trg_expense_after_update – dual-case balance adjustment

Table 13: Database Triggers for Real-time Balance Integrity

Trigger Name	Event	Table	Technical Effect
`trg_income_after_insert`	INSERT	Income	**Increments** the linked account balance by `NEW.Amount`.
`trg_expense_after_insert`	INSERT	Expenses	**Decrements** the linked account balance by `NEW.Amount`.
`trg_income_after_delete`	DELETE	Income	**Rolls back** the balance addition by subtracting `OLD.Amount`.
`trg_expense_after_delete`	DELETE	Expenses	**Rolls back** the balance deduction by adding `OLD.Amount`.
`trg_income_after_update`	UPDATE	Income	**Adjusts** balance: reverses old account/amount, applies new account/amount.
`trg_expense_after_update`	UPDATE	Expenses	**Adjusts** balance: restores old account/amount, deducts new account/amount.

2.5 Database Security

A dedicated MySQL user `finance_app` is created following the **Principle of Least Privilege (PoLP)**.

```
-- 1. Create restricted user
CREATE USER 'finance_app'@'localhost' IDENTIFIED BY 'FinApp@2025!';

-- 2. Global read-only access for reporting
GRANT SELECT ON personal_finance.* TO 'finance_app'@'localhost';

-- 3. Execute permissions for UDFs and stored procedures
GRANT EXECUTE ON personal_finance.* TO 'finance_app'@'localhost';

-- 4. Full CRUD on transactional tables
GRANT INSERT, UPDATE, DELETE ON personal_finance.Income
    TO 'finance_app'@'localhost';
GRANT INSERT, UPDATE, DELETE ON personal_finance.Expenses
    TO 'finance_app'@'localhost';

-- 5. Full CRUD on budgets (DELETE required for budget removal)
GRANT INSERT, UPDATE, DELETE ON personal_finance.Budgets
    TO 'finance_app'@'localhost';
```

```
19
20 -- 6. Profile and category management
21 GRANT INSERT, UPDATE ON personal_finance.Users
22     TO 'finance_app'@'localhost';
23 GRANT INSERT ON personal_finance.ExpenseCategories
24     TO 'finance_app'@'localhost';
25
26 -- 7. Account creation only -- no manual balance override
27 GRANT INSERT ON personal_finance.BankAccounts
28     TO 'finance_app'@'localhost';
29
30 FLUSH PRIVILEGES;
31 SHOW GRANTS FOR 'finance_app'@'localhost';
```

Listing 9: Security Configuration and Granular Permissions

Security Rationale:

- **No Structural DDL:** The user cannot execute `DROP`, `ALTER`, or `CREATE` statements, protecting the schema from destructive attacks.
- **Balance Integrity via Partial Access:** `UPDATE` is withheld for `BankAccounts`. All balance adjustments are handled exclusively by database triggers, preventing the application from manually falsifying records.
- **Budget DELETE Permission:** Unlike `Users` and `ExpenseCategories`, `Budgets` grants `DELETE` because users must be able to remove spending limits they no longer need. Deleting a budget carries no structural risk to other tables.
- **Restricted DELETE scope:** `DELETE` is granted only on `Income`, `Expenses`, and `Budgets` – never on `Users`, `BankAccounts`, or `ExpenseCategories`.

Table 14: Permission Matrix for `finance_app`

Table/Scope	**Privileges**	**Reason**
`personal_finance.*`	`SELECT, EXECUTE`	Allows reporting and running stored logic across all views, UDFs, and SPs.
`Income, Expenses`	`INSERT, UPDATE, DELETE`	Full CRUD; triggers handle all balance side-effects automatically.
`Budgets`	`INSERT, UPDATE, DELETE`	Full CRUD required: users must be able to create, adjust, and remove limits.
`Users`	`INSERT, UPDATE`	Allow profile creation and editing; `DELETE` withheld to prevent accidental account removal.
`BankAccounts`	`INSERT` only	Allow creating new accounts; `UPDATE` withheld so balance can only change via triggers.
`ExpenseCategories`	`INSERT` only	Allow users to add new spending categories dynamically; existing categories are never deleted via app.

3 Implementation and Functional Completeness

3.1 Project Structure

Listing 10: Project Directory Structure

```
personal-finance/
+-- database/
|   +-- schema.sql        # CREATE TABLE (6 tables, constraints, FKs)
|   +-- sample_data.sql   # Large-scale dataset
|   +-- objects.sql       # Indexes, Views, UDFs, SPs, Triggers
|   +-- security.sql      # Restricted user + privilege grants
+-- app/
|   +-- db_connection.py # Centralised connection factory
|   +-- models.py         # All CRUD and query functions
|   +-- reports.py        # Matplotlib chart generators (3 charts)
|   +-- main.py           # CustomTkinter GUI
+-- screenshot/           # UI screenshots for report
+-- ERD.png               # Entity-Relationship Diagram
+-- Topic.pdf             # Original project brief
```

3.2 Python Module Responsibilities

Table 15: Python Module Summary

Module	Responsibility
`db_connection.py`	Provides a single `get_connection()` factory function using `mysql-connector-python`. Centralising the connection ensures that changing the host, database name, or credentials requires editing exactly one file. `autocommit=False` so that transactions are explicit.
`models.py`	Contains all database interaction functions organised by entity (Users, BankAccounts, Income, Expenses, Budgets, ExpenseCategories). All queries use parameterised placeholders (`%s`) to prevent SQL injection. Full CRUD is implemented for `Income` (`add`, `update`, `delete`), `Expenses` (`add`, `update`, `delete`), and `Budgets` (`set`, `delete`). Additional helpers provide daily, monthly, and yearly financial summaries.
`reports.py`	Generates three `matplotlib.figure.Figure` objects: (1) side-by-side bar chart of monthly income vs expenses; (2) exploded pie chart of category spending with a legend showing absolute VND amounts; (3) filled line chart of cumulative net balance using `numpy.cumsum()`. All three charts include a hidden annotation object for tooltip support activated in `main.py`.
`main.py`	Single-window CustomTkinter application with a sidebar and four screen frames (`DashboardScreen`, `TransactionsScreen`, `ReportsScreen`, `BudgetsScreen`). Navigation raises the selected frame; each screen implements a `refresh()` method called on activation to reload data from the database. Interactive chart tooltips are implemented via `mpl_connect('motion_notify_event')` callbacks registered against the hidden annotation objects.

3.3 Key Implementation Patterns

3.3.1 Parameterised Query Helper

Rather than opening a connection in every function, a private helper `execute_query()` in `models.py` handles connection lifecycle, commit/rollback, and cursor cleanup:

```
def execute_query(query, params=None, fetch=False, many=False):
  conn = get_connection()
  if not conn:
    return None
  try:
    with conn.cursor(dictionary=True) as cursor:
      cursor.execute(query, params or ())
      if fetch:
        result = cursor.fetchall() if many else cursor.fetchone()
      else:
        conn.commit()
        result = cursor.rowcount  # number of affected rows
```

```
      return result
  except Error as e:
    if conn:
      conn.rollback()
    raise e
  finally:
    if conn:
      conn.close()
```

Listing 11: execute_query helper in models.py

Note: For queries containing `DATE_FORMAT` with `%Y-%m`, direct connection objects are used instead of this helper to avoid double-escaping of the `%` character by `mysql-connector`.

3.3.2 Trigger-Driven Balance Update

The most important business rule is enforced entirely at the database layer. The Python `add_income()` and `add_expense()` functions insert a row and commit; the corresponding trigger fires automatically:

```
def add_expense(user_id, category_id, account_id,
                amount, expense_date, description):
    """
    Insert an expense record.
    The Trigger (trg_expense_after_insert) automatically deducts
    the amount from BankAccounts.Balance.
    """
    return execute_query(
        """INSERT INTO Expenses (UserID, CategoryID,
           AccountID, Amount, ExpenseDate, Description)
           VALUES (%s, %s, %s, %s, %s, %s)""",
        (user_id, category_id, account_id, amount,
         expense_date, description)
    )
```

Listing 12: add_expense function in models.py

3.3.3 Full CRUD with Trigger-Aware Updates

Beyond simple insert operations, `models.py` implements `update` and `delete` functions for both `Income` and `Expenses`. These operations are fully trigger-aware: the database triggers handle all balance adjustments automatically, so the Python layer requires no additional balance logic. The update triggers handle two distinct cases: (1) when only the amount changes on the same account, and (2) when the account itself changes.

```
def update_income(income_id, account_id, amount,
                  date, description):
    """ Update an income record.
    The trigger handles balance delta automatically."""
    query = """
        UPDATE Income
        SET AccountID = %s, Amount = %s,
        IncomeDate = %s, Description = %s
        WHERE IncomeID = %s
```

```
    """
    return execute_query(
        query, (account_id, amount, date, description, income_id))

def delete_expense(expense_id):
    """Delete an expense record.
    trg_expense_after_delete restores the account balance."""
    query = "DELETE FROM Expenses WHERE ExpenseID = %s"
    return execute_query(query, (expense_id,))
```

Listing 13: update_income and delete_expense in models.py

3.3.4 Daily and Yearly Summary Queries

Beyond monthly reports, `models.py` provides two additional time-granularity helpers to satisfy FR-11:

```
def get_daily_summary(user_id):
    """Return today's total income and expenses."""
    query = """
        SELECT
            (SELECT COALESCE(SUM(Amount), 0)
                FROM Income
                WHERE UserID=%s
                AND IncomeDate = CURDATE()) as day_in,
            (SELECT COALESCE(SUM(Amount), 0)
                FROM Expenses
                WHERE UserID=%s
                AND ExpenseDate = CURDATE()) as day_out
    """
    return execute_query(query, (user_id, user_id), fetch=True)


def get_yearly_summary(user_id, year):
    """Return total income and expenses for a full year."""
    query = """
        SELECT
            COALESCE(SUM(TotalIncome), 0) as yr_in,
            COALESCE(SUM(TotalExpenses), 0) as yr_out
        FROM vw_monthly_summary
        WHERE UserID = %s AND MonthYear LIKE %s
    """
    return execute_query(
        query, (user_id, f"{year}-%"), fetch=True)
```

Listing 14: Daily and yearly summary functions in models.py

`get_daily_summary` uses correlated subqueries with `CURDATE()` to avoid a full table scan. `get_yearly_summary` leverages the existing `vw_monthly_summary` view with a `LIKE` pattern, reusing the aggregation logic already defined at the database layer.

3.4 Application Screens

3.4.1 Dashboard

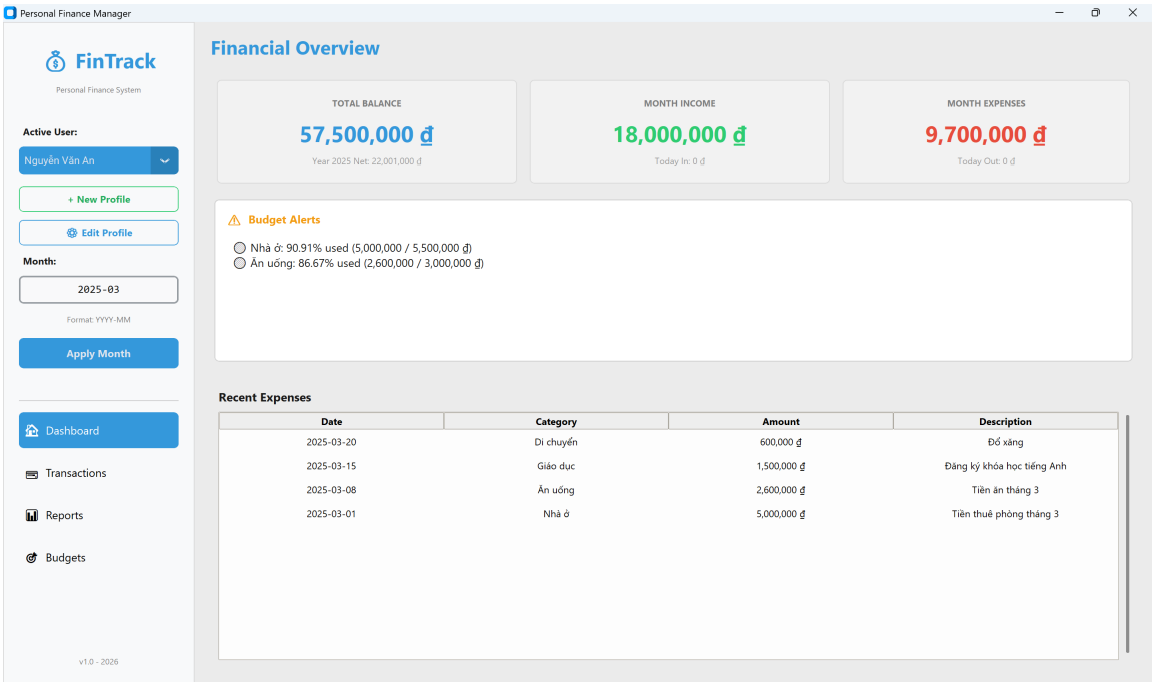


Figure 3: Dashboard screen showing three KPI cards (Total Balance, Month Income, Month Expenses), each with a secondary sub-metric: yearly net savings (*Year 2025 Net*), today's income (*Today In*), and today's expenses (*Today Out*). The Budget Alerts panel lists categories at WARNING or EXCEEDED status from `vw_budget_status`. The Recent Expenses table shows the latest transactions for the selected month. User: Nguyen Van An, Month: 2025-03.

The Dashboard queries four data sources on every `refresh()` call:

- `get_total_balance(user_id)` – sums `BankAccounts.Balance`.
- `get_yearly_summary(user_id, year)` – yearly net displayed as a sub-label under Total Balance.
- `get_daily_summary(user_id)` – today's income and expense totals shown as sub-labels under Month Income and Month Expenses.
- `get_budget_alerts(user_id, month_year)` – queries `vw_budget_status` for WARNING and EXCEEDED rows only.

3.4.2 Transactions – Add Income

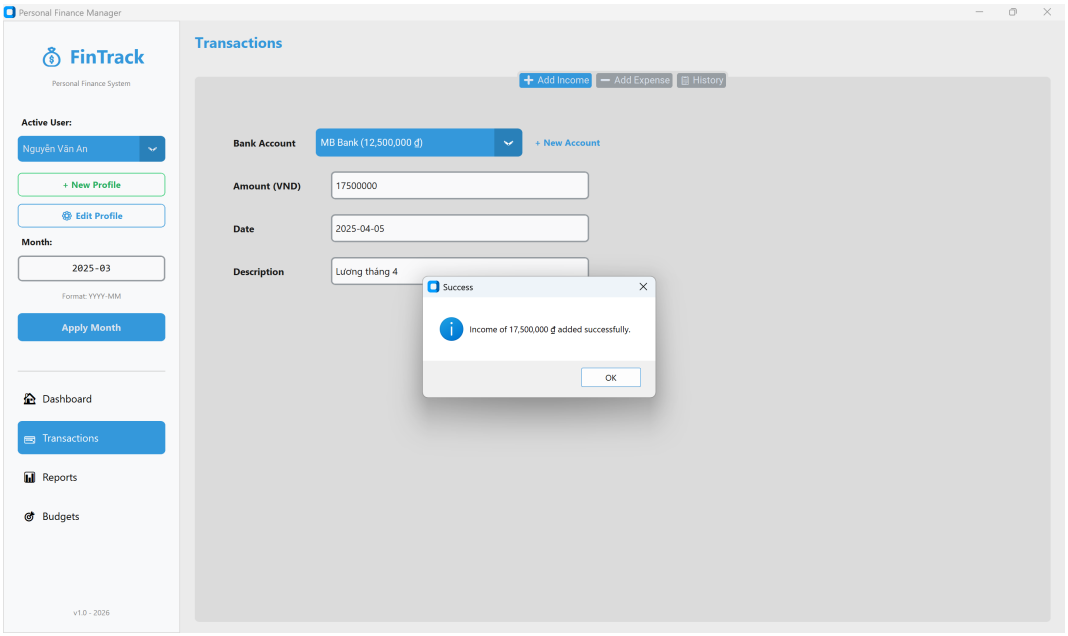


Figure 4: Add Income tab: user enters 17,500,000 VND into MB Bank for 2025-04-05. The success dialog confirms the record was saved and the bank balance was incremented automatically by `trg_income_after_insert`. The Transactions screen provides three tabs: Add Income, Add Expense, and History.

3.4.3 Transactions – Add Expense and Add New Category

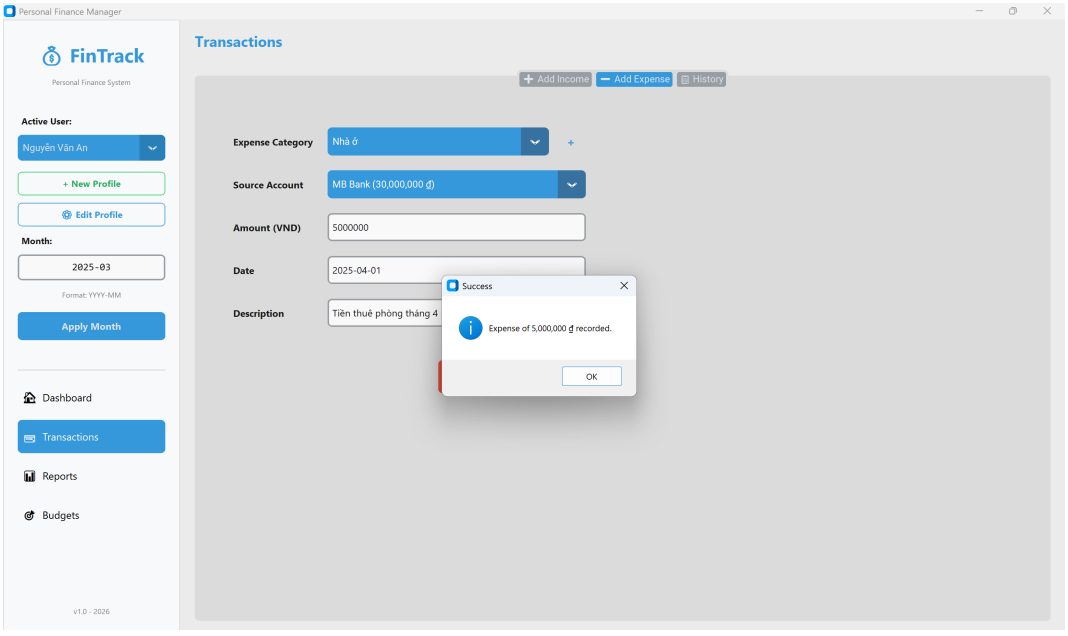


Figure 5: Add Expense tab: user records a 5,000,000 VND housing expense (Nha o) debited from MB Bank. The trigger `trg_expense_after_insert` immediately decremented the account balance upon commit.

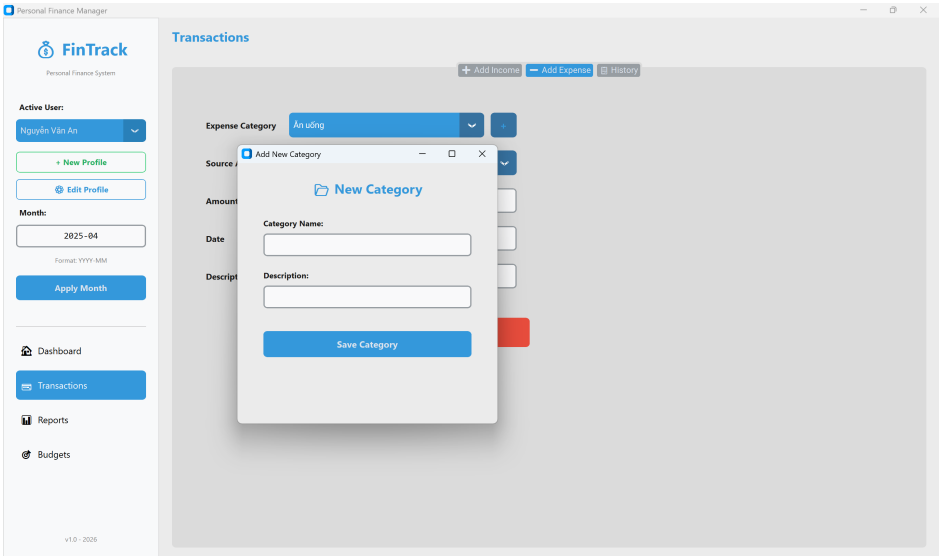


Figure 6: Add New Category dialog, opened from the + button beside the Expense Category dropdown. The user enters a Category Name and optional Description. Clicking Save Category calls `create_category()`, which inserts a row into `ExpenseCategories` and returns the new `CategoryID`. The parent dropdown refreshes immediately to include the new entry.

3.4.4 Transactions – History, Edit, and Delete

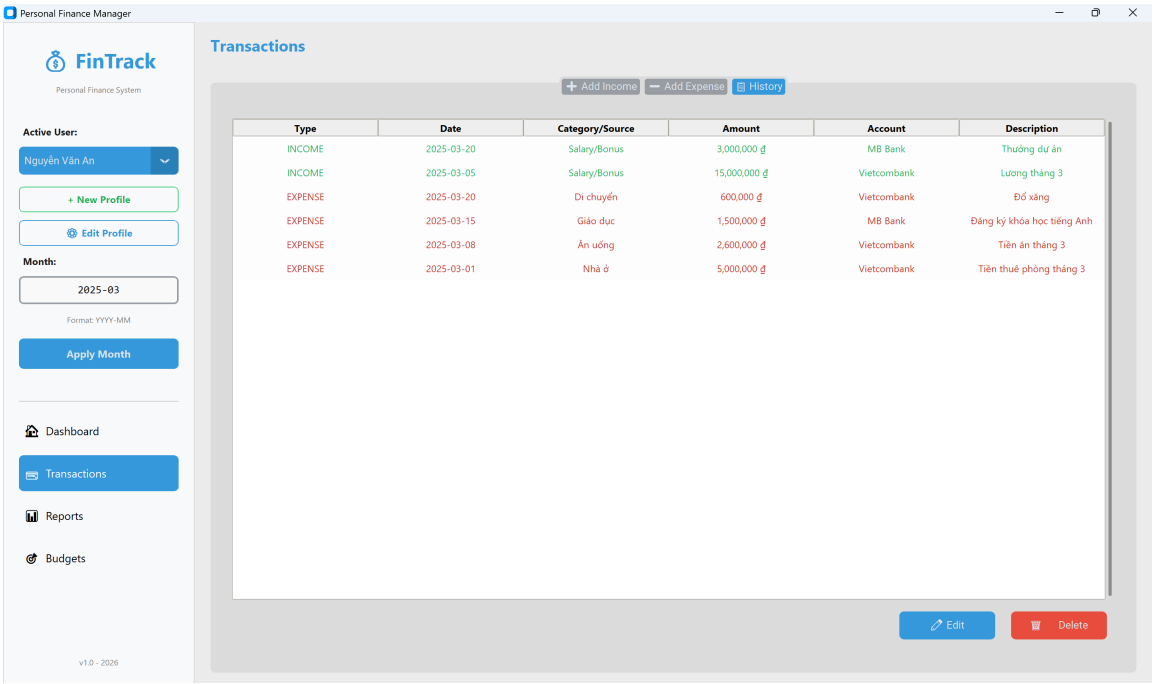


Figure 7: History tab showing all income (green) and expense (red) records for the selected month in a unified table. Columns include Type, Date, Category/Source, Amount, Account, and Description. Selecting a row enables the Edit and Delete buttons at the bottom right.

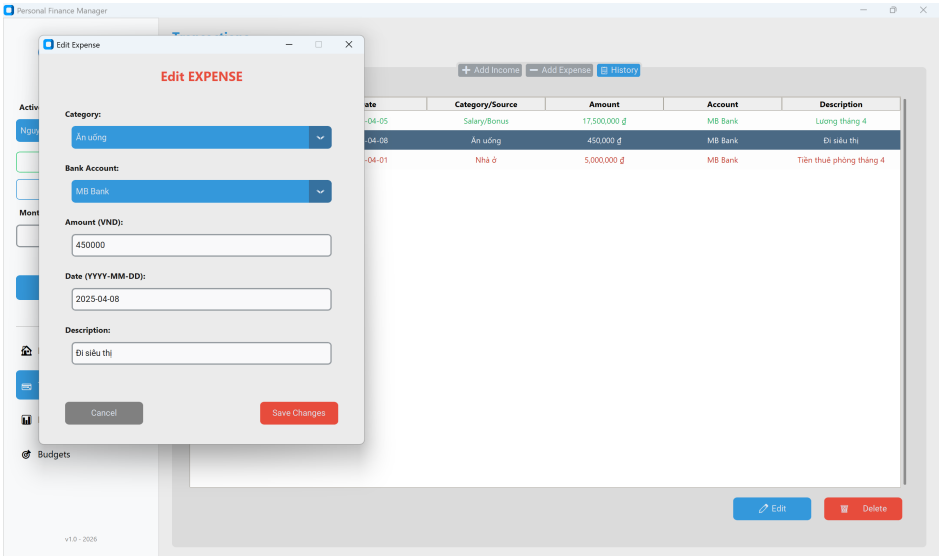
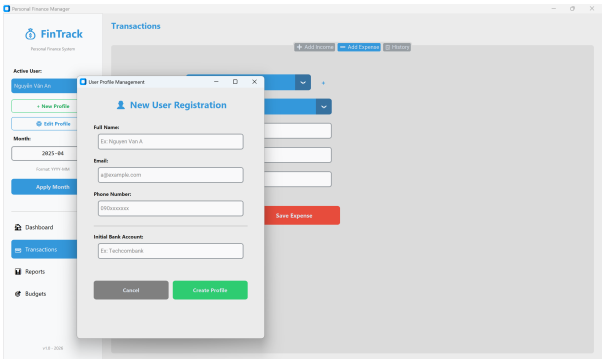
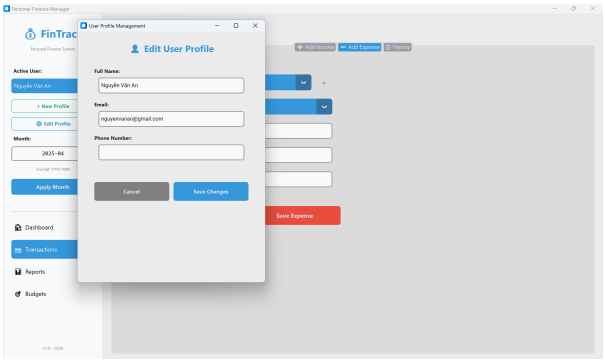


Figure 8: Edit Expense dialog, opened by selecting a row in the History tab. All fields are pre-populated with the existing record's values. Clicking Save Changes calls `update_expense()`; the trigger `trg_expense_after_update` automatically adjusts the account balance – applying only the delta if the amount changed on the same account, or transferring between accounts if the Bank Account field was changed. The same modal pattern applies to income records via `update_income()` and `trg_income_after_update`.

3.4.5 User Profile – New Profile and Edit Profile



(a) New User Registration dialog. Collects Full Name, Email, Phone Number, and Initial Bank Account name. Clicking Create Profile calls `create_user()` followed by `create_account()` to register the user and their first bank account in sequence.



(b) Edit User Profile dialog, pre-populated with the active user's current data. Clicking Save Changes calls `update_user_profile()`, issuing a parameterised `UPDATE` on the `Users` table.

Figure 9: User Profile Management dialogs, accessible from the + **New Profile** and **Edit Profile** buttons in the sidebar, available on all screens.

3.4.6 Budget Management

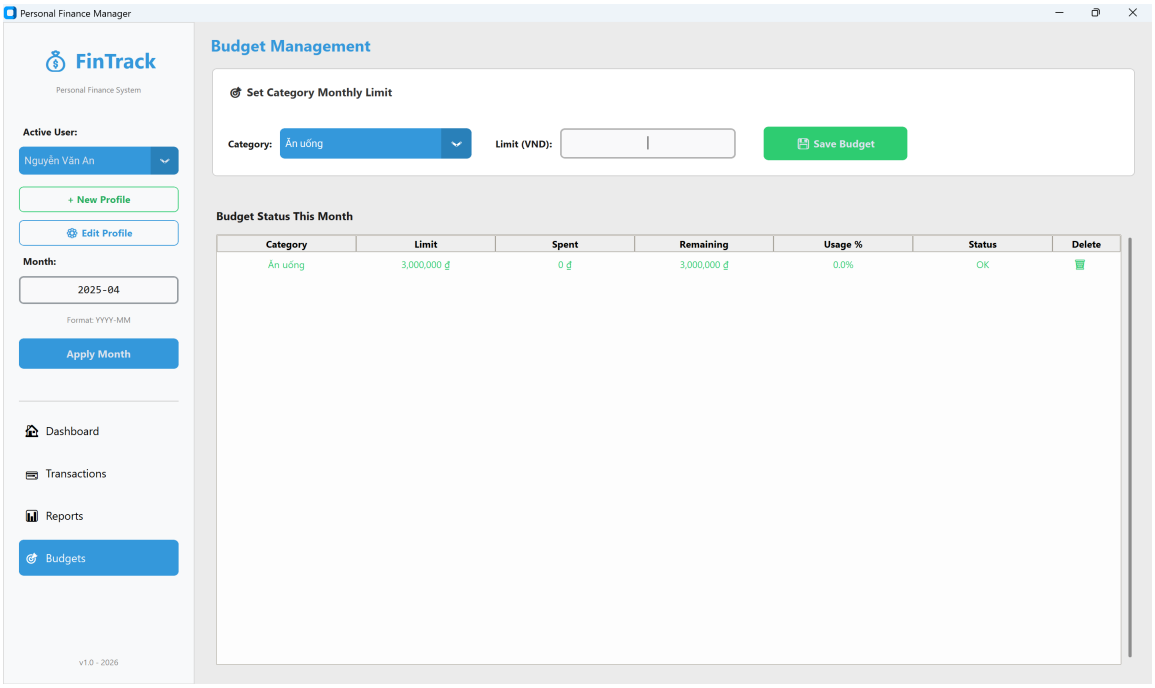


Figure 10: Budget Management screen: a 3,000,000 VND limit is set for An uong (Food & Dining) in 2025-04. The Budget Status table reads from `vw_budget_status` and displays Category, Limit, Spent, Remaining, Usage%, and Status. Each row has a trash-can icon in the Delete column for inline budget removal.

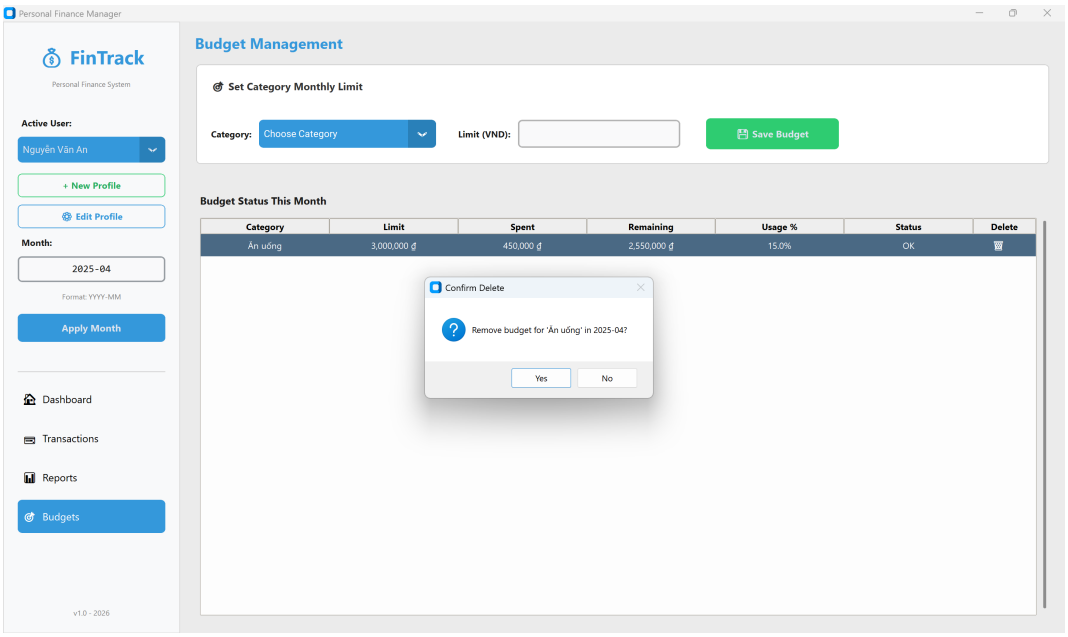


Figure 11: Budget deletion confirmation dialog. The dialog shows the category name and month to prevent accidental deletion. Confirming calls `delete_budget(user_id, category_id, month_year)`, which issues a `DELETE` against the `Budgets` table. The Budget Status table and Dashboard alerts panel both refresh automatically on the next screen activation.

3.5 Reports and Analytics

The Reports screen is divided into two tabs: **Performance Snapshots** for numerical summaries and **Visual Analytics** for interactive charts.

3.5.1 Performance Snapshots

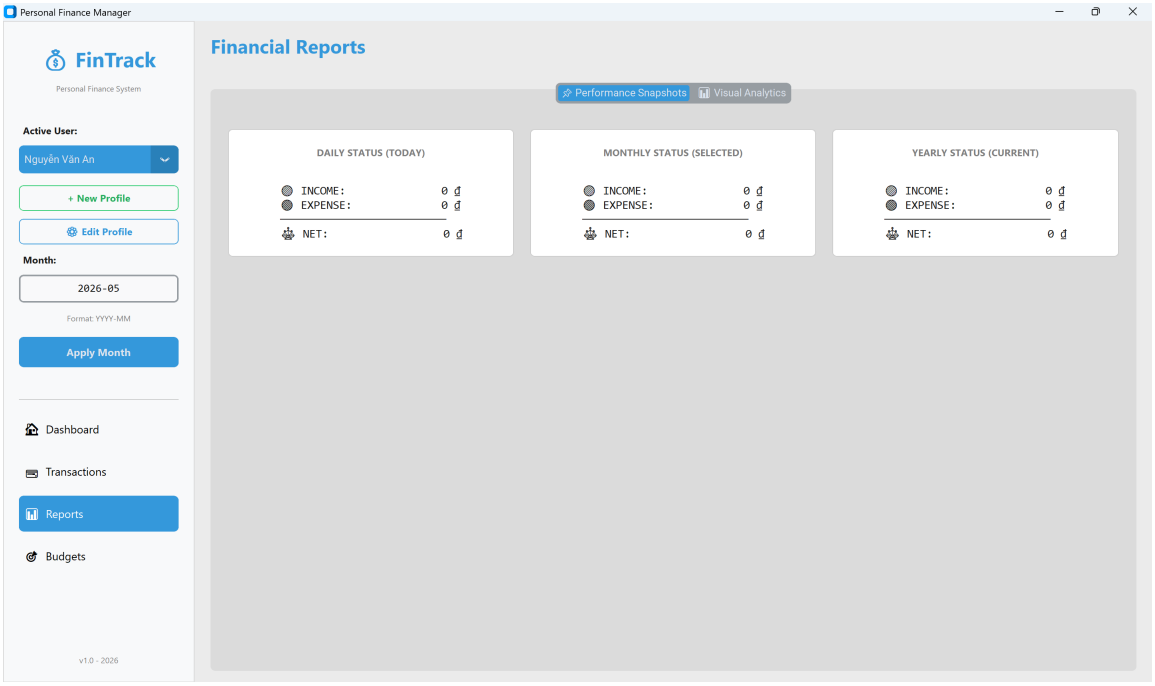


Figure 12: Performance Snapshots tab showing three summary cards side-by-side: Daily Status (today via `CURDATE()`), Monthly Status (selected month from `vw_monthly_summary`), and Yearly Status (current year aggregated from `vw_monthly_summary` using a `LIKE` pattern). Each card shows Income, Expense, and Net figures.

Table 16: Performance Snapshot Cards and their Data Sources

Card	Python Function	SQL Mechanism
Daily Status (Today)	`get_daily_summary()`	Correlated subqueries with `CURDATE()` on `Income` and `Expenses` tables.
Monthly Status (Selected)	`get_monthly_summary()`	Query on `vw_monthly_summary` filtered by selected month.
Yearly Status (Current)	`get_yearly_summary()`	Aggregation on `vw_monthly_summary` with `LIKE 'YYYY-%'` pattern.

3.5.2 Visual Analytics – Monthly Income vs Expenses Bar Chart

Figure 13: Income vs Expenses bar chart for months 2025-01 through 2025-04. Income (green) consistently exceeds expenses (red) across all months, indicating positive net savings. An interactive tooltip (visible above the 2025-02 income bar: *Income: 20,000,000 d*) appears on mouse hover, implemented via a `motion_notify_event` callback in `main.py` that makes a hidden `matplotlib` annotation object visible when the cursor enters a bar region.

3.5.3 Visual Analytics – Spending by Category Pie Chart

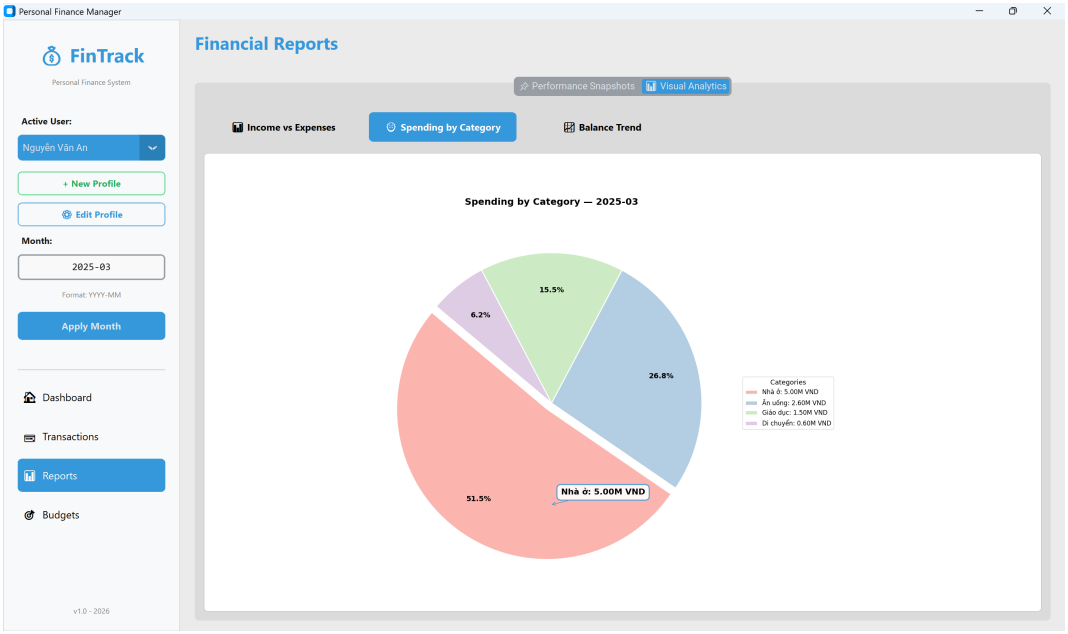


Figure 14: Category spending distribution for 2025-03. Nha o (Housing) dominates at 51.5% and is slightly exploded for visual emphasis. The interactive tooltip shows the exact VND amount on hover (*Nha o: 5.00M VND*). The legend lists all categories with their absolute amounts in VND.

3.5.4 Visual Analytics – Net Balance Trend Line Chart

Figure 15: Cumulative net balance trend from 2025-01 to 2025-04, reaching 35M VND. An interactive tooltip shows the exact balance at each data point on hover (*Balance: 22,001,000 d*). The chart is built using a `UNION ALL` subquery that merges all income and expense events into a single timeline, then applies `numpy.cumsum()` to produce the running balance. The shaded area under the line and the zero-reference dashed line provide immediate visual context for whether cumulative savings are positive.

3.6 Sample Data

The database was populated with a large-scale, realistic Vietnamese financial dataset to ensure system performance and verify the robustness of triggers and reporting logic:

Table 17: Sample Data Summary

Table	Records	Notes
Users	50	Diverse user base with varied professions
BankAccounts	100	Average of 2 accounts per user (Cash and Banking)
ExpenseCategories	15	Comprehensive categories (Food, Transport, etc.)
Income	194	Multi-year monthly salaries and freelance records
Expenses	444	Detailed daily transaction history (2025)
Budgets	323	Monthly category-wise spending limits across users

4 Analysis, Evaluation and Discussion

4.1 Functional Verification

Each requirement was verified through manual testing:

Table 18: Functional Requirement Test Results

ID	Feature	Test Action	Result
FR-03	Income trigger	Inserted 17,500,000 VND; balance incremented correctly	**PASS**
FR-04	Expense trigger	Inserted 5,000,000 VND; balance decremented correctly	**PASS**
FR-05	Budget CRUD	Set, updated, and deleted a 2,000,000 VND Food budget	**PASS**
FR-06	Budget alert	95% Food usage → WARNING displayed on Dashboard	**PASS**
FR-07	Monthly report	`vw_monthly_summary` returns correct aggregates	**PASS**
FR-08	Stored proc.	`CALL sp_monthly_report(1,'2025-01')` returns 2 result sets	**PASS**
FR-09	UDF	`SELECT fn_get_total_expenses (1,'2025-01')` = 10,970,000	**PASS**
FR-10	GUI navigation	All 4 screens load without errors; data refreshes on activation	**PASS**
FR-11	Daily/yearly	Dashboard shows today's figures; yearly aggregation correct	**PASS**
FR-12	Income/Expense CRUD	Edit and delete income/expense; balance updated by trigger	**PASS**
FR-13	Category & profile	New category added inline; user profile updated without error	**PASS**

4.2 Trigger Correctness Verification

The trigger mechanism was verified directly in MySQL Workbench:

```
-- Step 1: Record initial balance
SELECT AccountID, BankName, Balance
FROM BankAccounts WHERE AccountID = 1;
-- Result: Vietcombank | 45,000,000.00

-- Step 2: Insert income
INSERT INTO Income
(UserID, AccountID, Amount, IncomeDate, Description)
VALUES (1, 1, 1000000, '2025-04-01', 'Trigger test');

-- Step 3: Verify balance incremented
SELECT Balance FROM BankAccounts WHERE AccountID = 1;
-- Result: 46,000,000.00  (+1,000,000 -- trigger fired correctly)

-- Step 4: Insert expense
```

```sql
INSERT INTO Expenses
(UserID, CategoryID, AccountID, Amount, ExpenseDate, Description)
VALUES (1, 1, 1, 500000, '2025-04-01', 'Trigger test');

-- Step 5: Verify balance decremented
SELECT Balance FROM BankAccounts WHERE AccountID = 1;
-- Result: 45,500,000.00  (-500,000 -- trigger fired correctly)
```

Listing 15: Trigger Verification Queries

4.3 Evaluation Against Requirements

4.3.1 What Worked Well

1. **Full lifecycle trigger integrity:** All six triggers correctly maintained balance consistency across INSERT, UPDATE, and DELETE scenarios. The UPDATE triggers handle both same-account amount changes and cross-account transfers using a dual-case `IF/ELSEIF` structure. Because the logic lives in MySQL, the invariant holds even if the database is accessed directly through MySQL Workbench or the CLI.
2. **Budget alert system:** The `vw_budget_status` view encapsulates a non-trivial multi-table aggregation with conditional logic (`CASE` expression). The Python layer simply queries the view; adding a new status tier (e.g. 90% "CRITICAL") would require only a change to the view definition.
3. **Separation of concerns:** No SQL string exists in `main.py`. The four-file architecture cleanly separates presentation, business logic, reporting, and data access. This made debugging significantly easier – a chart rendering bug and a balance display bug were both isolated and fixed without touching the database layer.
4. **Realistic sample data:** Salaries, rent, grocery, and transport amounts were calibrated against Vietnamese living-cost statistics, making the system credible for demonstration purposes.
5. **Interactive tooltips on all charts:** All three matplotlib charts implement hover tooltips via a hidden `annotate` object that becomes visible when the mouse enters a bar, wedge, or data point. This was implemented entirely in `main.py` using `mpl_connect('motion_notify_event')` without modifying `reports.py`, demonstrating clean separation between chart generation and UI interaction logic.
6. **Dual-granularity reporting:** The Performance Snapshots tab provides daily, monthly, and yearly figures simultaneously, satisfying the requirement for multiple financial summary granularities without requiring additional database objects beyond the existing `vw_monthly_summary` view and a `CURDATE()` subquery.

4.3.2 Limitations and Known Issues

1. **MySQL % escaping:** `mysql-connector-python` escapes all `%` characters in the parameterised `cursor.execute()` path. Queries using `DATE_FORMAT(col, '%Y-%m')` inside the generic `execute_query` helper produced `'%%Y-%%m'` at runtime. The fix was to bypass the helper for affected queries and use direct connection objects. A cleaner long-term solution would be to replace `DATE_FORMAT` with `YEAR(col)` and `MONTH(col)`.

2. **No authentication layer:** User selection is handled via a dropdown. A production system would require bcrypt-hashed passwords and a login screen.
3. **Single-machine deployment:** The application connects to `localhost` only. A shared deployment would require a networked MySQL instance with TLS encryption.
4. **Generic error messages on constraint violation:** If the `BankAccounts CHECK` constraint rejects an expense, the application displays a generic error. A more robust implementation would catch MySQL error code 3819 and display a user-friendly "Insufficient balance" message.
5. **No concurrency control:** The helper opens a new connection per call, which is safe for single-user desktop use. A multi-user deployment would require connection pooling and explicit transaction isolation levels.

4.4 Comparison: Database-Layer vs Application-Layer Business Logic

Table 19: Database Layer vs Application Layer Trade-offs

Concern	Chosen Layer	Rationale
Balance update after transaction	MySQL Trigger	Guaranteed regardless of access method
Budget usage calculation	MySQL View + UDF	Single source of truth; no duplication
Monthly report aggregation	MySQL Stored Proc	Complex SQL kept in one place
Chart rendering	Python (matplotlib)	Database cannot generate visual output
UI event handling	Python (tkinter)	Desktop GUI requires application code
Input validation (format)	Python	Faster feedback before DB round-trip

4.5 Future Improvement Directions

1. **Savings goals module:** Add a `SavingsGoals` table (`GoalName`, `TargetAmount`, `Deadline`) and a trigger that checks remaining balance against active goals after each expense.
2. **Exchange rate API integration:** Connect to an open exchange-rate API to support multi-currency income with automatic conversion to VND at the date of receipt.
3. **PDF report export:** Use the `reportlab` library to export the monthly financial report as a professionally formatted PDF.
4. **Email / push notifications:** Send a monthly summary email using Python's `smtplib` when a budget reaches 80% or 100%.
5. **Predictive analytics:** Apply a simple linear regression on the `vw_monthly_summary` data to project next month's expenses by category, alerting users in advance if a budget overrun is likely.

5 Report Quality, Presentation and Academic Standards

5.1 Deliverables Checklist

Table 20: Project Deliverables

Deliverable	Status	Location
ER Diagram	✓ Complete	ERDPlus export (PNG)
`schema.sql`	✓ Complete	`database/schema.sql`
`sample_data.sql`	✓ Complete	`database/sample_data.sql`
`objects.sql`	✓ Complete	`database/objects.sql`
`security.sql`	✓ Complete	`database/security.sql`
`db_connection.py`	✓ Complete	`app/db_connection.py`
`models.py`	✓ Complete	`app/models.py`
`reports.py`	✓ Complete	`app/reports.py`
`main.py`	✓ Complete	`app/main.py`
GitHub repository	✓ Complete	`https://github.com/phngan23/Personal_Finance_Management_System`
YouTube demo video	✓ Complete	`https://youtube.com/YOUR_LINK`
This report (PDF)	✓ Complete	Submitted via LMS

5.2 Technology Stack Summary

Table 21: Technology Stack

Component	Technology
Database server	MySQL 8.0
Database design tool	MySQL Workbench 8.0
Programming language	Python 3.x
DB connector library	mysql-connector-python 8.x
GUI framework	CustomTkinter 5.x
Chart library	Matplotlib 3.x
Chart embedding	FigureCanvasTkAgg (TkAgg backend)
Numerical library	NumPy
IDE	Visual Studio Code
Version control	Git / GitHub

6 Conclusion

This project successfully delivered a fully functional Personal Finance Management System that meets all stated functional requirements. The key technical achievement is the consistent use of MySQL's advanced features – triggers, views, stored procedures, and UDFs – to enforce business logic at the database layer, ensuring data integrity regardless of how the database is accessed.

The three-layer Python architecture (presentation / business logic / data access) demonstrates good software engineering practice: the UI never writes SQL, the database layer never handles GUI concerns, and the reporting layer is independently testable.

All four graphical screens (Dashboard, Transactions, Reports, Budgets) operate correctly on the sample dataset. The full lifecycle trigger suite – covering INSERT, UPDATE, and DELETE on both Income and Expenses – was independently verified through direct MySQL queries, including the dual-case UPDATE scenario where funds move between accounts. The two bugs encountered during development (MySQL `%` escaping and JOIN-based view data loss) were diagnosed, understood at a technical level, and resolved with justified fixes.

The system provides a solid foundation for the five future improvements described in Section 4.4, the most impactful of which – predictive analytics and mobile banking API integration – would substantially increase the system's practical value for everyday Vietnamese users.

References

[1] MySQL 8.0 Reference Manual – Triggers.
https://dev.mysql.com/doc/refman/8.0/en/triggers.html

[2] MySQL 8.0 Reference Manual – Stored Routines (Procedures and Functions).
https://dev.mysql.com/doc/refman/8.0/en/stored-routines.html

[3] MySQL 8.0 Reference Manual – CREATE VIEW Statement.
https://dev.mysql.com/doc/refman/8.0/en/create-view.html

[4] MySQL 8.0 Reference Manual – CREATE INDEX Statement.
https://dev.mysql.com/doc/refman/8.0/en/create-index.html

[5] MySQL 8.0 Reference Manual – Access Control and Account Management.
https://dev.mysql.com/doc/refman/8.0/en/access-control.html

[6] MySQL 8.0 Reference Manual – GRANT Statement.
https://dev.mysql.com/doc/refman/8.0/en/grant.html

[7] MySQL 8.0 Reference Manual – CREATE FUNCTION Statement for UDFs.
https://dev.mysql.com/doc/refman/8.0/en/create-function.html

[8] MySQL 8.0 Reference Manual – INSERT ... ON DUPLICATE KEY UPDATE.
https://dev.mysql.com/doc/refman/8.0/en/insert-on-duplicate.html

[9] mysql-connector-python Developer Guide.
https://dev.mysql.com/doc/connector-python/en/

[10] mysql-connector-python – `cursor.callproc()` Method for Stored Procedures.
https://dev.mysql.com/doc/connector-python/en/connector-python-api-mysqlcursor-callproc.html

[11] CustomTkinter Documentation – GUI Framework by Tom Schimansky.
https://customtkinter.tomschimansky.com/documentation/

[12] Matplotlib Documentation – Pyplot Tutorial for Data Visualization.
https://matplotlib.org/stable/tutorials/introductory/pyplot.html

[13] Matplotlib Documentation – Event Handling and Picking (`mpl_connect`).
https://matplotlib.org/stable/users/explain/figure/event_handling.html

[14] Matplotlib Documentation – Interactive Annotations and Tooltips.
https://matplotlib.org/stable/tutorials/text/annotations.html

[15] Matplotlib – `FigureCanvasTkAgg` for Embedding Charts in Tkinter.
https://matplotlib.org/stable/api/backend_tk_api.html

[16] NumPy Documentation – `numpy.cumsum` for Cumulative Net Savings.
https://numpy.org/doc/stable/reference/generated/numpy.cumsum.html